

Projeto e Análise de Algoritmos

Seleção em tempo linear

Thiago Pinheiro de Araújo

EMAp/FGV

2025.2

Apresentar **algoritmos de seleção em tempo linear**, discutir as características de cada um, analisar a complexidade, e exibir exemplos de implementação.

Seleção em tempo linear

O problema

- Dada uma sequência não-ordenada com n elementos distintos:

```
int v[] = {8, 11, 2, 5, 10, 16, 7, 15, 1, 4};
```

- Problema: como retornar a mediana do conjunto sem ordenar a sequência?
 - Se a sequência estivesse ordenada bastaria fazer:
 - Se n for ímpar: $v_{\frac{n}{2}}$
 - Se n for par: $\frac{v_{\frac{n}{2}} + v_{\frac{n}{2}-1}}{2}$

O problema

- **Problema geral:** como retornar o i -ésimo elemento sem ordenar a sequência?
- Se pudéssemos ordenar a sequência bastaria retornar o elemento v_i
 - A complexidade seria $\Theta(n \log(n))$, considerando um algoritmo de ordenação eficiente.
- Outra opção seria percorrer a sequência k vezes buscando pelo menor valor e removendo-o
 - A complexidade é $\Theta(kn)$.
 - Se for buscada a mediana teremos $k = n/2$, logo uma complexidade $\Theta(n^2)$.

Quickselect

Quickselect

- O algoritmo **Quickselect** consiste em particionar a sequência conforme o algoritmo Quicksort, e buscar recursivamente escolhendo uma das partições.
- Dada a sequência $v[0..n-1]$ e a posição x buscada
 - Executamos o algoritmo de particionamento, obtendo o pivô j .
 - As partições esquerda e direita terão tamanho $|L| = j$ e $|R| = n - j$
 - Se $|L| = x - 1$, encontramos o elemento.
 - Se $|L| > x - 1$, executamos a recursão para $[0..j-1]$.
 - Se $|L| < x - 1$, executamos a recursão para $[j+1..n-1]$.

Seleção em tempo linear

Quickselect

- Exemplo: vamos buscar o terceiro menor elemento da sequência abaixo:

8	11	2	5	1	16	4	15	6	7
---	----	---	---	---	----	---	----	---	---

2	6	1	4	5	7	16	11	15	8
---	---	---	---	---	---	----	----	----	---

2	6	1	4	5
---	---	---	---	---

2	1	4	5	6
---	---	---	---	---

2	1	4
---	---	---

Encontrou!



Seleção em tempo linear

Quickselect

- Exemplo de implementação:

```
int quickselect(int v[], int l, int r, int x) {  
    if (x > 0 && x <= r - l + 1) {  
        int j = partition(v, l, r);  
        if (j - l == x - 1) {  
            return v[j];  
        }  
        if (j - l > x - 1) {  
            return quickselect(v, l, j - 1, x);  
        }  
        return quickselect(  
            v,  
            j + 1,  
            r,  
            x - j + l - 1);  
    }  
    return -1;  
}
```


- Vamos **analisar o desempenho** do algoritmo Quickselect.
 - No melhor caso a sequência será particionada usando a mediana como pivô, levando à seguinte função de recorrência:

$$T(n) = cn + T\left(\frac{n}{2}\right)$$

- Resolvendo a recorrência:

$$\begin{aligned} T(n) &= cn + T\left(\frac{n}{2}\right) \\ &= cn + \frac{cn}{2} + T\left(\frac{n}{2}\right) \\ &= cn + \frac{cn}{2} + \frac{cn}{4} + T\left(\frac{n}{4}\right) \\ &= cn \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{n} \right) \leq O(n) \end{aligned}$$

- Vamos **analisar o desempenho** do algoritmo Quickselect.
 - No entanto, no pior caso a sequência será particionada gerando sempre um conjunto vazio e um conjunto com $n - 1$ elementos, levando à seguinte função de recorrência:

$$T(n) = cn + T(n - 1)$$

- Resolvendo a recorrência:

$$\begin{aligned} T(n) &= cn + T(n - 1) \\ &= cn + c(n - 1) + T(n - 2) \\ &\dots \\ &= c(n + (n - 1) + (n - 2) + \dots + 2 + 1) \\ &= \frac{n(n + 1)}{2} \\ &= O(n^2) \end{aligned}$$

Quickselect

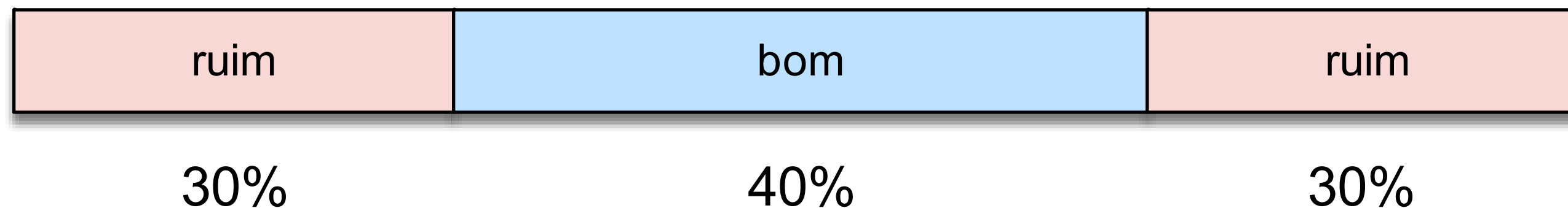
- **Conclusão:** assim como no algoritmo Quicksort, a complexidade depende da escolha do pivô.
- Se conseguirmos escolher sempre a mediana da sequência em $O(n)$, garantimos que Quickselect também será $O(n)$.

Mediana das Medianas

Seleção em tempo linear

Mediana das medianas

- O algoritmo **Mediana das medianas** é semelhante ao Quickselect, no entanto utiliza uma estratégia para escolher o pivô de forma que as partições sejam balanceadas.
- Utilizando essa abordagem o particionamento produz no pior caso conjuntos com tamanho $\frac{3n}{10}$ e $\frac{7n}{10}$.



Seleção em tempo linear

Mediana das medianas

- Dada a sequência $v[0..n-1]$ e a posição x buscada
 - Divida a sequência em grupos de 5 elementos.
 - Ordene cada grupo e obtenha a mediana de cada um deles (força bruta)
 - Insira cada mediana em uma sequência M .
 - Encontre a mediana m das medianas M executando o algoritmo recursivamente.
 - Particione a sequência utilizando a mediana m , obtendo o pivô j .
 - Se $|L| = x - 1$, encontramos o elemento.
 - Se $|L| > x - 1$, executamos a recursão para $[0..j-1]$.
 - Se $|L| < x - 1$, executamos a recursão para $[j+1..n-1]$.

Seleção em tempo linear

Mediana das medianas

- Vamos buscar a mediana do conjunto a seguir:

$v = [1, 4, 12, 17, 54, 36, 13, 90, 83, 2, 19, 15, 72, 5, 8, 21, 44, 68, 51, 25, 35, 76, 92, 41, 61]$

- Divida em grupos de 5:

G1	G2	G3	G4	G5
1	36	19	21	35
4	13	15	44	76
12	90	72	68	92
17	83	5	51	41
54	2	8	25	61

Seleção em tempo linear

Mediana das medianas

- Vamos buscar a mediana do conjunto a seguir:

$v = [1, 4, 12, 17, 54, 36, 13, 90, 83, 2, 19, 15, 72, 5, 8, 21, 44, 68, 51, 25, 35, 76, 92, 41, 61]$

- Encontre a mediana de cada grupo:

G1	G2	G3	G4	G5
1	2	5	21	35
4	13	8	25	41
12	36	15	44	61
17	83	19	51	76
54	90	72	68	92

Seleção em tempo linear

Mediana das medianas

- Vamos buscar a mediana do conjunto a seguir:

$v = [1, 4, 12, 17, 54, 36, 13, 90, 83, 2, 19, 15, 72, 5, 8, 21, 44, 68, 51, 25, 35, 76, 92, 41, 61]$

- Encontre a mediana das medianas:

G1	G3	G2	G4	G5
1	5	2	21	35
4	8	13	25	41
12	15	36	44	61
17	19	83	51	76
54	72	90	68	92

Seleção em tempo linear

Mediana das medianas

■ Exemplo de implementação:

```
int selectMOM(int v[], int p, int r, int k) {
    int n = r - p + 1;
    if (k <= 0 || k > n) { return -1; }
    int median[(n + 4) / 5];
    int i = 0;
    int pos = p;
    while (pos <= r) {
        int size = r - pos + 1;
        median[i++] = medianOf(
            &v[pos],
            (size >= 5) ? 5 : size);
        pos += 5;
    }
    int mom = (i == 1) ?
        median[i-1] :
        selectMOM(median, 0, i - 1, i / 2);
    int j = partition(v, p, r, mom);
    if (j - p == k - 1) { return v[j]; }
    if (j - p > k - 1) {
        return selectMOM(v, p, j - 1, k);
    }
    return selectMOM(v, j + 1, r, k - j + p - 1);
}
```

```
int medianOf(int v[], int n) {
    quicksort(v, 0, n - 1);
    return v[n / 2];
}
```

Mediana das medianas

- Vamos analisar porque o particionamento irá gerar no pior caso conjuntos com $3n/10$ e $7n/10$ elementos.
- A metade dos $n/5$ grupos terá garantidamente uma mediana menor que a mediana das medianas (MOM)
 - Logo, $\frac{n/5}{2} = \frac{n}{10}$ grupos.
- Cada grupo possui pelo menos 3 elementos menores que a mediana das medianas
 - Portanto existem ao menos $\frac{n}{10} * 3 = \frac{3n}{10}$ elementos menores que MOM.
 - Logo, no pior caso a chamada recursiva será realizada sobre $\frac{7n}{10}$ elementos.

- Vamos **analisar o desempenho** do algoritmo a partir da função de recorrência:

$$T(n) = \begin{cases} c_1 & n = 1 \\ T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n) & n > 1 \end{cases}$$

- Estamos buscando que $T(n)$ seja linear, logo $T(n) \leq cn$:

$$\begin{aligned} T(n) &\leq \frac{cn}{5} + \frac{7cn}{10} + c_1n & \left(\frac{9c}{10} + c_1\right)n &\leq cn \\ &= \frac{9cn}{10} + c_1n & c &= 10c_1 \\ &= \left(\frac{9c}{10} + c_1\right)n \end{aligned}$$

- **Exercício:** Uma empresa de análise de redes sociais, que possui uma base de dados com milhões de usuários, está desenvolvendo um algoritmo para identificar influenciadores emergentes. Além de informações cadastrais, em cada usuário é armazenado o seu número de seguidores e uma pontuação representando o seu índice de engajamento, calculado com base em uma fórmula envolvendo likes, comentários e compartilhamentos. Um influenciador emergente é um usuário que está no top 10% em termos de engajamento, porém não está no top 10% em número de seguidores. Projete um algoritmo eficiente que, dada uma sequência A com n tuplas $\langle id_usuário, seguidores, engajamento \rangle$, identifique todos os influenciadores emergentes. O algoritmo deverá apresentar uma complexidade $O(n)$.

■ Solução:

```
vector<User> findEmergingInfluencers(vector<User>& users) {  
    size_t n = users.size();  
    size_t k_engagement = n * 0.9;  
    size_t k_followers = n * 0.9;  
    float engagement_cutoff = selectMOM(users, k_engagement);  
    float followers_cutoff = selectMOM(users, k_followers);  
  
    vector<User> emergingInfluencers;  
    for (const auto& user : users) {  
        if (user.engagement > engagement_cutoff && user.followers <= followers_cutoff) {  
            emergingInfluencers.push_back(user);  
        }  
    }  
  
    return emergingInfluencers;  
}
```

```
struct User {  
    int id;  
    int followers;  
    float engagement;  
};
```

Perguntas?

`thiago.p.araujo@fgv.br`